

1.9. Funciones

En muchas ocasiones nos encontraremos ante un script de una cierta dimensión o complejidad, el cual lo podemos dividir en partes más pequeñas, de forma que, cada una sirve para un determinado propósito, creándose de esta manera las funciones.

Las funciones nos van a permitir **ejecutar** una serie de instrucciones **cuando se produce un evento** (por ejemplo, cuando **se pulsa un botón**). Se pueden definir tantas funciones como sea preciso.



Evento

Un evento es una acción del usuario ante la cual puede realizarse algún proceso (por ejemplo, el **cambio del valor de un formulario o la pulsación de un enlace**).

También nos permitirán que se ejecute el **mismo** código sin necesidad de insertar la secuencia de instrucciones repetidas veces.



Funciones

Una función es un **conjunto de instrucciones a las que se asigna un nombre, devuelve o no un valor y se puede ejecutar tantas veces** como se desea con solo llamarla con el nombre asignado.

A la hora de trabajar con funciones debemos distinguir dos aspectos:

- la **creación** de la función o definición de la misma y
- la **utilización** de la función o llamada a esta.

1.9.1. Creación de una función

Para crear una función debemos determinar los siguientes aspectos:

- El **nombre** de la función.
- Los **parámetros** o valores sobre los que **actuará**, es decir, los **argumentos**.
- Las **acciones** que deberá realizar.
- El **valor** que **devolverá**, aunque **puede** no devolver **ningún** valor.

Y aplicar el siguiente formato:

```
function nombreFuncion (listaArgumentos)
{
    instrucciones...
    ...
    [return valorRetorno;]
}
```

Dónde:

- `nombreFuncion` será el identificador válido que nos servirá para realizar las llamadas a la función.
- `listaArgumentos` será la lista de variables (**separadas por comas**) que recogerán los valores que se **pasen** en la llamada a la función.
- `valorRetorno` será la expresión cuyo valor **devolverá** la función.

Vamos a ver un pequeño ejemplo de una función que la llamamos `suma`, la cual recibe dos valores `(A,B)` y devuelve su suma:

```
function suma (A,B)
{
    var C;
    C = A + B;
    return C;
}
```

Debemos tener en cuenta, que cuando el **navegador** encuentra la definición de una función, la va a **cargar** en la memoria, pero **no la va a ejecutar hasta que se produzca una llamada a dicha función**.

1.9.2. ¿Dónde declarar una función?

Las funciones pueden declararse en cualquier parte de una página HTML entre las etiquetas `<script>` y `</script>` pero siempre debemos tener en cuenta que no pueden definirse:

- Dentro de otra función.
- Dentro de una estructura de control.(if, for, etc...)

Normalmente se definen dentro de la **cabecera** de una página HTML, así cuando se **carga** la página en el navegador del cliente estarán **disponibles** para ser utilizadas.

Vamos a ver un ejemplo de función con la que visualizaremos la hora en formato Hora:Minutos.

```
<script type="text/javascript">
function Hora()
{
```

```

        var hoy =new Date()
        document.write(hoy.getHours(),":",hoy.getMinutes())
    }
</script>

```

Podemos observar en el anterior ejemplo, que la función se llama `Hora()`, no recibe argumentos y no devuelve ningún valor. Define una variable `hoy` que utiliza el objeto `Date()` para poder obtener la hora `hoy.getHours()` y los minutos `hoy.getMinutes()`. Este objeto lo trataremos en el capítulo de objetos.

1.9.3. Llamada a una función

Ya sabemos cómo y dónde declarar una función. Para realizar las llamadas que producirán la ejecución de la misma como si se tratase de cualquier instrucción, deberemos hacerlas siguiendo el formato:

`nombreFuncion(listaParametros);`

Debemos tener en cuenta que la **llamada** a una función **nos puede devolver un valor** y que debemos hacer algo con él, ya sea escribirlo, asignarlo a una variable, etcétera.

Por ejemplo, para hacer una llamada a la función `suma` y escribir el valor devuelto en el documento actual escribiríamos lo siguiente:

```
document.write(suma(2,3));
```

Por ejemplo, para llamar a la función `Hora` sólo tendremos que escribir su nombre y los paréntesis vacíos porque no lleva argumentos `Hora()`

```

<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Funciones</title>
    <script type="text/javascript">
        function Hora()
        {
            var hoy =new Date();
            document.write(hoy.getHours(),":",hoy.getMinutes());
        }
    </script></head>
<body >
    <script type="text/javascript">
        document.write("Hora actual: ");
        Hora();
    </script>
</body>
</html>

```

1.9.4. Argumentos



Argumentos

Son variables locales a la función que toman un valor determinado cuando se les invoca.

Las funciones con argumentos generan resultados **distintos** dependiendo de los valores que tomen los argumentos cuando se llama a la función. La función `suma (A,B)` expuesta a continuación recibe dos valores y devuelve su suma:

```
<head>
<title>Funciones</title>

<script type="text/javascript">
    function suma (A,B) {
        var C;
        C = A + B;
        return C;
    }
</script>
</head>
<body >
    <script type="text/javascript">
        document.write("Llamada a la función Suma con argumentos:
    ");
        document.write(suma(2,3));
    </script>
</body>
</html>
```

Cuando lo ejecutemos obtendremos el siguiente resultado:

Llamada a la función Suma con argumentos: 5

1.9.5. Consideraciones con el uso de funciones

Cuando usamos funciones en *JavaScript*, debemos tener en cuenta los siguientes aspectos:

- Que tanto las **funciones** como las **llamadas** deben ir **entre** etiquetas `<script> </script>`.
- Que la cláusula `return` produce la **salida** de la función devolviendo (opcionalmente) un valor.



Debemos recordar que **JavaScript** permite escribir **varias** cláusulas `return` en una misma función.

```
function mayor(A,B)
{
    if (A >= B) {
        return A;
    }
    else {
        return B;
    }
}
```

- Debemos tener en cuenta que **se pueden escribir funciones que no devuelvan ningún valor**, sino que **solamente** realizan una o varias **acciones**. También se pueden escribir funciones que **no tengan parámetros**.

Si usamos funciones con parámetros, y le **pasamos un número menor de parámetros**, estos que no han obtenido valor como resultado de la llamada quedarán con el valor `null`. Por ejemplo, si la llamada a la función suma se realiza con un solo número el resultado será:

Llamada a la función Suma con argumentos: NaN

Para solucionar esto, podemos **comprobar el número de parámetros que se ha pasado a una función** mediante `arguments.length` tal como podemos apreciar en el siguiente ejemplo:

```
<script type="text/javascript">
function suma(A,B)
{
    var C;
    if (arguments.length < 2) {
        B = 0;
    }
    if (arguments.length < 1) {
        A = 0;
    }
    C = A + B;
    return C;
}
</script>
```

1.9.6. Variables locales y globales

Debemos saber diferenciar que son las variables locales y globales. Son variables **locales** aquellas que **se declaran en la definición de la función**, y **sólo** son **accesibles** en el **ámbito de la función** donde se declaran.

Las variables **globales** se **declaran fuera de la declaración de cualquier función** y se puede tener **acceso** a ellas desde **cualquier parte** del documento.

```

<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Funciones</title>

    <script type="text/javascript">
        var C; //variable global,
        C=0;
        function suma(A,B) {
            C = A + B;
            return C;
        }
        function resta(A,B) {
            return C+A-B;
        }
    </script>
</head>
<body >
    <script type="text/javascript">
        document.write("Sumamos dos valores: ");
        document.write(suma(2,3));
        document.write('<BR>');
        document.write("Restamos dos valores: ");
        document.write(resta(10,20));
    </script>
</body>
</html>

```

Si ejecutamos el script, obtenemos la siguiente salida:

```

Suma con argumentos: 5

Restamos dos valores -5

```

1.9.7. Funciones predefinidas

Existen algunas funciones propias del lenguaje que nos van a permitir convertir cadenas a enteros o reales, evaluar una expresión, etc.; aunque algunas de ellas ya las hemos visto en puntos anteriores, volveremos a definirlas y explicarlas en este apartado:

- **parseFloat(cadena):** esta función nos **devuelve un número en coma flotante** a partir de la cadena especificada. La función **finalizará la conversión cuando encuentre un carácter que no sea un dígito (0-9), un punto decimal (.) o una "e"**. Ejemplos:

```

parseFloat("27.11.2011"); //devuelve 27.11
parseFloat("20H30M07S"); //devuelve 20
parseFloat("345.6e5"); //devuelve 34560000
parseFloat("VUELO506"); //devuelve NaN (Not a Number)

```

- **parseInt(cadena):** nos devolverá un **número entero** a partir de la cadena especificada. La función finalizará la conversión cuando encuentre un carácter que no sea un dígito (0-9). Ejemplos:

```
parseInt("27.11.2000"); //devuelve 27
parseInt("20H30M07S"); //devuelve 20
parseInt("345.6e5"); //devuelve 345
parseInt("VUELO506"); //devuelve NaN (Not a Number)
```

- **parseInt(cadena, base):** si **no se indica nada** al respecto, se considerará como **base** la **decimal** (excepto cuando el número comienza con 0x o con 0); pero en este formato se puede añadir un segundo argumento que indicaría la base en la que creemos que está el número que queremos convertir. Ejemplos:

```
parseInt("27", 8); //devuelve 23
parseInt("1111111", 2); //devuelve 255
parseInt("A2", 16); //devuelve 162
```

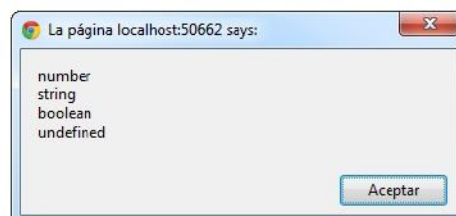
- **toString(argumento):** convierte **a cadena el dato especificado** en argumento. Normalmente se utiliza para convertir **números a cadena** pero también funciona con otros tipos como Booleano, etcétera.

Su utilización es restringida ya que *JavaScript* aplica una conversión automática de tipos a cadena cuando encuentra expresiones de concatenación en las que hay diversos tipos de datos.

- **typeof(argumento):** nos devolverá una cadena que indica el tipo del argumento (number, string, boolean, undefined). El siguiente programa muestra la utilización de esta función:

```
<script type="text/javascript">
  V1 = 10;
  V2 = 'a';
  V3 = true;
  var V10;
  alert(typeof(V1)+ "\n"+ typeof(V2)+ "\n"+
  typeof(V3)+ "\n"+ typeof(V10));
</script>
```

Si ejecutamos el script nos dará lo siguiente:



- **escape(cadena):** devuelve la secuencia de códigos de escape de cada uno de los caracteres de la cadena que le pasemos como argumento. Estos códigos serán devueltos con el formato %NN, donde **NN es el código de escape en hexadecimal** del carácter.

- **unescape(cadena):** devuelve la cadena formada por los caracteres cuyo código le especificaremos. Los códigos deben ser especificados con el mismo formato con el que escape los proporciona: %NN, donde NN es el código en hexadecimal.
- **isNaN(argumento):** con esta función evaluaremos el argumento que se le pasa y **nos devolverá false si es un número**; en caso contrario, devuelve true. Por ejemplo:


```
isNaN("seas"); devuelve true.  
isNaN("10"); devuelve false.
```
- **eval(expresión):** devuelve el resultado de evaluar la expresión pasada como parámetro. Por ejemplo `eval("2+3");` nos devuelve 5.

1.9.8. Funciones de cadena de texto

Cuando hablamos de las cadenas de texto, debemos saber que JavaScript nos provee a través del objeto ***String*** un conjunto de funciones que van a permitir al programador multitud de operaciones con las cadenas de texto.

Aunque ya se han visto algunas de ellas durante este temario, vamos a verlas todas juntas

charAt

La función *charAt* la vamos a usar para conocer **cuál** es el **caracter** que se encuentra en la **posición** índice de la cadena.

```
caracter = cadena.charAt(índice);
```

Los índices, al igual que en las matrices, comienzan en 0.

```
caracter = "seas".charAt(3);
```

Esta sentencia nos devolverá en la variable `caracter` el valor "a", que es el que ocupa la posición 3.

Length

Utilizaremos la función *length* para conocer la **longitud** de una cadena.

```
longitud = "seas".length;
```

La salida de esta sentencia es que la variable `longitud` tiene un valor de 4.

IndexOf

Utilizaremos la función *indexOf* para saber **si una cadena está contenida en otra**, y si es así, nos devuelve la posición en la cadena de búsqueda donde se inicia la contención.

```
posicion = "estudiosabiertos".indexOf("os");
```

La salida de esta sentencia es que la variable `posicion` tiene un valor de 7.

Si no se encuentra la subcadena dentro de la cadena donde se busca, nos devolverá el **valor -1**.

Además, esta función tiene un segundo parámetro, que es opcional, y que sirve para indicar la posición **a partir de la cual** se ha de buscar la subcadena.

```
posicion = "estudiosabiertos".indexOf("os",3);
```

En este ejemplo se comenzaría a buscar desde la posición 3, es decir, desde la "t".

Substr

Usaremos la función *substr* para obtener un **subconjunto** de la cadena de texto.

```
cadena = "Texto".substr(inicio,longitud);
```

Un ejemplo sería:

```
cadena = "estudiosabiertos".substr(3,5);
```

Empezando desde el carácter de la posición 3 y con una **longitud** de 5 caracteres, la salida de esta sentencia es que `cadena` tendrá un valor de "udios".

Slice

La función *slice* es muy similar a la función *substr* descrita en el punto anterior, pero la diferencia es que el **segundo** argumento en lugar de ser la longitud de la subcadena a extraer **es la posición del último carácter a extraer, sin incluirlo**. Así, el ejemplo anterior sería como sigue:

```
cadena = "estudiosabiertos".slice(3,5);
```

En este caso es el mismo rango de números. El carácter que ocupa la posición 5 es "i", y como no se incluye ese carácter, la cadena resultante sería "ud".

El segundo parámetro es opcional, y si no se pone, se llega hasta el final de la cadena.

Substring

Es otra función más para obtener una subcadena a partir de otra. Funciona exactamente igual a *slice* pero el segundo parámetro sí que es obligatorio. Esta función, junto con *substr*, es de las más usadas.

```
cadena = "estudiosabiertos".substring(3,5);
```

toLowerCase

Esta función se utiliza para convertir a **minúsculas** la cadena a la que aplica:

```
minusculas = "EstudiosAbiertoS".toLowerCase();
```

El resultado será que `minusculas` será igual a `"estudiosabiertos"`.

toUpperCase

Función que se utiliza para convertir a **mayúsculas** la cadena a la que aplica.

```
mayusculas = "EstudiosAbiertoS".toUpperCase();
```

El resultado será que `mayusculas` será igual a `"ESTUDIOSABIERTOS"`.

Estas dos funciones son de uso frecuente en las comparaciones entre cadenas, ya que normalmente no nos interesarán las diferencias entre mayúsculas o minúsculas.